Homework 4: Library Checkout System 📖 !

For this assignment, you will be implementing methods that allow patrons to check out and return books from a library system. You will also be writing and fixing test cases to ensure that all checkout and return operations are handled correctly and that the system maintains accurate records of library activity.

Review the starter code thoroughly before beginning this assignment, as understanding how the classes interact with each other is important. Take notes or draw a diagram if necessary.

Overview

This assignment consists of three classes: **Book**, **Patron**, and **Library**. Most of the starter code has already been provided for you. You will complete specific methods as outlined in the **Tasks to Complete** section.

Book Class

The entire **Book** class has been provided for you (DO NOT CHANGE any of this code).

Each **Book** object has the following instance variables:

- **book_id** - an integer representing the book's unique ID
- **title** - a string representing the book's title
- **author** - a string representing the book's author
- **is_checked_out** - a boolean value representing whether the book is currently checked out (**True** is checked out, **False** otherwise)
- **checkout_history** - a list of patron objects representing the patrons who have previously checked out this book

The **Book** class also includes several methods:

- ***__init__*** - initializes the book's attributes
- ***__str__*** - returns a string representation of the book
- ***has_been_borrowed_by(self, patron_id)*** - returns **True** if the patron with the given ID has borrowed this book, otherwise returns **False**

Note:

- The checkout_history should be updated only when a book is successfully checked out (See the Library Class's ***checkout_book(self, patron_id, book_id)*** for more information)

Patron Class

The **Patron** class represents a library member who can check out and return books. The entire Patron class has been provided for you (DO NOT CHANGE any of this code).

Each **Patron** object has the following instance variables:

- **patron_id** - an integer representing the patron's unique ID
- **name** - a string representing the patron's name
- **checked_out_books** - a list of Book objects that the patron **currently** has checked out

The **Patron** class includes the following methods:

- ***__init__*** - initializes the patron's attributes
- ***__str__*** - returns a string representation of the patron

Note:

- A patron may check out **at most 4 books at a time**.

Library Class

The **Library** class manages all books, patrons, and transactions. Most of the Library class has been provided for you.

(Only implement the **checkout_book**, **return_book**, **count_successful_checkouts**, and [optionally] **return_all_books** functions - Do NOT modify anything else in this class. See the **Tasks to Complete** section below for more information)

Each Library object has the following instance variables:

- **name** - a string representing the library name
- **books** - a list of Book objects
- **patrons**- a list of Patron objects
- **transaction_log** - a list of tuples that records all checkout and return attempts. Each tuple in the transaction log has the following format and order: *(action, patron_id, book_id, status)*, where:

    - ***action*** is a string that must be either "checkout" or "return"
    - ***patron_id*** is an integer
    - ***book_id*** is an integer
    - ***status*** is a string describing the outcome of the operation. It must be one of "success", "not_found", "unavailable", "at_limit", or "not_checked_out". See the Tasks to Complete section for exactly when each value applies.

The **Library** class includes the following methods:

- ***__init__*** - initializes the library's attributes
- ***__str__*** - returns a string representation of the library
- ***add_book(self, book)*** - adds a **Book** object to the library's list of books if it is not already present; returns **True** if the book was added and **False** otherwise
- ***add_patron(self, patron)*** - adds a **Patron** object to the library's list of patrons if it is not already present; returns **True** if the patron was added and **False** otherwise
- ***checkout_book(self, patron_id, book_id)*** - attempts to check out the book with the given **book_id** to the patron with the given **patron_id**; returns **True** if the checkout was successful and **False** otherwise
- ***return_book(self, patron_id, book_id)*** - attempts to return the book with the given **book_id** for the patron with the given **patron_id**; returns **True** if the return was successful and **False** otherwise
- ***count_successful_checkouts(self)*** - returns a dictionary mapping each **patron_id** to the number of times that patron has successfully checked out a book, based on the library's **transaction_log**

See the **Tasks to Complete** section below for additional details on checkout and return logic.

Tasks to Complete

➔ Complete the Library Class

- ***checkout_book(self, patron_id, book_id)*** - completes the checkout process for a book. Takes **patron_id** (int) and **book_id** (int) and returns a boolean value (True or False) that indicates whether the checkout was successful.
    - A checkout should only succeed if all of the following conditions are met. Check the conditions in the order listed below, and if more than one of them fails, record the status for the first one that fails:
        - a patron with the given **patron_id** exists in the library's patrons list. If not, the status recorded is "not_found".

        - a book with the given **book_id** exists in the library's books list. If not, the status recorded is "not_found".
        - the book with the given **book_id** is not currently checked out. If the book is already checked out, the status recorded is "unavailable".
        - the patron with the given **patron_id** has fewer than 4 books checked out. If the patron is already at the limit, the status recorded is "at_limit".
    - If the checkout is successful:
        - add the book to the patron's **checked_out_books** attribute
        - update the book's **checkout_history** attribute
        - update the book's **is_checked_out** attribute
        - record the transaction in the library's **transaction_log** as a successful checkout, using "success" as the status.
    - If the checkout is not successful:
        - do not modify the patron or book state
        - record the transaction in the library's **transaction_log** as an unsuccessful checkout, using the status that corresponds to the condition that failed.
- ***return_book(self, patron_id, book_id)*** - completes the return process for a book. Takes **patron_id** (int) and **book_id** (int) and returns a boolean value (True or False) that indicates whether the return was successful.
    - A return should only succeed if all of the following conditions are met. Check the conditions in the order listed below, and if more than one of them fails, record the status for the first one that fails:
        - a patron with the given **patron_id** exists in the library's patrons list. If not, the status recorded is "not_found".
        - a book with the given **book_id** exists in the library's books list. If not, the status recorded is "not_found".
        - the book is currently checked out by the patron with the given **patron_id**. If not, the status recorded is "not_checked_out".
    - If the return is successful:
        - remove the book from the patron's **checked_out_books**
        - update the book's **is_checked_out** attribute
        - record the transaction in the library's **transaction_log** as a successful return, using "success" as the status
    - If the return is not successful:
        - do not modify the patron or book state
        - record the transaction in the library's **transaction_log** as an unsuccessful return, using the status that corresponds to the condition that failed

- ***count_successful_checkouts(self)*** - counts the number of successful checkouts made by each patron. Returns a dictionary mapping each **patron_id** to the number of successful checkouts that patron made, based on the library's **transaction_log**.
    - The dictionary returned must be in the following format:
        - {*patron_id: count*}
    - A transaction should be counted only if its action is "checkout" and its status is "success". Only patrons with at least one successful checkout may appear in the dictionary: a patron with no successful checkouts must be left out entirely, rather than included with a count of 0. If there are no successful checkout transactions at all, return an empty dictionary

➔ Write a Test Case

Note: Many test cases have already been written for you. **DO NOT edit any test cases other than the following:**

- ***test_library_return_book_not_checked_out_by_patron*** - The book was not checked out by the patron. Complete this test case to cover the following scenario *exactly*:
    - A patron with **patron_id = 2** successfully checks out a book with **book_id 101**.
    - A different patron with **patron_id = 1** then attempts to return that same book.
    - Verify that the **return_book** method returns **False**
    - Verify that the most recent entry in transaction_log records this attempt as an unsuccessful return with the status "not_checked_out"

Testing tip: You may comment out other test cases while developing a function or the above test case, but **be sure to uncomment all test cases before submitting your homework.**

Extra Credit (6 points)

For extra credit, complete ***return_all_books(self, patron_id)***. This method attempts to return **all books currently checked out** by the given patron.

The method should:

- First verify that **patron_id** exists in the library's list of patrons.
- If the patron does not exist, return **False** and do not modify any state.
- If the patron exists:
    - Attempt to return each book currently checked out by that patron
    - For each return attempt, record a transaction in **transaction_log** using the same format as ***return_book***
    - Remove all successfully returned books from the patron's **checked_out_books**

- Return **True** if at least one book was successfully returned; otherwise return **False**

Grading Rubric (60 points + 6 bonus points)

Note: *If you use hardcoding (specifying expected values directly) in any of the methods by way of editing to get them to pass the test cases, or you edit any test cases other than the ones you have been directed to, you will NOT receive credit for those related portions.*

Item	***Points***
checkout_book() (25 points)	
Correctly verifies that the specified patron exists in the library before attempting a checkout, and records the "not_found" status when the patron does not exist.	3 points
Correctly verifies that the specified book exists in the library before attempting a checkout, and records the "not_found" status when the book does not exist.	3 points
Correctly checks the book's current checkout status when processing a checkout request, and records the "unavailable" status when the book is already checked out.	3 points
Correctly enforces the rule that a patron may have at most four books checked out at a time, and records the "at_limit" status when the patron is already at that limit.	8 points
Correctly completes a checkout by updating all required state and recording a transaction with the "success" status.	8 points
return_book() (15 points)	
Correctly verifies that the specified patron exists in the library before attempting a return, and records the "not_found" status when the patron does not exist.	3 points
Correctly verifies that the specified book exists in the library before attempting a return, and records the "not_found" status when the book does not exist.	3 points
Correctly processes a return request by ensuring the book is checked out by the given patron, recording the "not_checked_out" status when	9 points

it is not, and updating all required state and transaction records when the return succeeds.	
count_successful_checkouts() (10 points)	
Correctly computes the number of successful checkouts per patron using the transaction log and returns the result in the required dictionary format.	7 points
Correctly handles cases with no successful checkout transactions by returning an empty dictionary, and omits patrons who have no successful checkouts.	3 points
test_library_return_book_not_checked_out_by_patron() (10 points)	
Correctly implements the required unit test for returning a book that was not checked out by the given patron	10 points
return_all_books() (bonus 6 points)	
Correctly verifies that the specified patron exists in the library before attempting to return any books.	1 points
Correctly handles the case where a valid patron has no books currently checked out.	2 points
Correctly processes a patron with checked-out books by attempting to return each book, updating all required patron and book state, recording transactions, and returning an appropriate boolean result.	3 points